2026 Visual Perception and 3D Computer Vision class

基于计算机视觉的实时魔法对战系统

Spells Battle 项目报告

张子健 俞鸿彬 陈秋澎

2026-6-26

目录

摘要 3

1 相关背景与技术选择 3

2 系统总体架构 4

3 视觉输入路径 5

3.1 Finger 模式：以食指尖作为最小可行输入 5

3.2 Wand 模式：亮色标记魔杖输入 7

3.3 Duel 模式下的多玩家输入归属 7

4 亮色魔杖追踪管线 8

4.1 检测与标定 8

4.2 ROI、候选与评分 8

4.3 光流、滤波与状态机 8

4.4 v2 诊断与改进过程 9

5 手势分类算法 9

5.1 几何特征启发式分类 9

5.2 DTW 模板与 shadow 评估 10

5.3 FusedGestureClassifier 融合策略 11

6 CV Training 数据闭环 11

7 对战逻辑与玩法系统 12

8 渲染表现与交互反馈 13

9 工程实现与可维护性 15

9.1 从单体脚本到模块化结构 15

9.2 参数集中与兼容回退 15

9.3 工具链 16

10 测试与验证 16
11 实验结果与分析 17
11.1 实时性 17
11.2 魔杖鲁棒性 17
11.3 分类融合策略 18
12 已知不足与未来工作 18
13 结论 19
参考文献 19
附录 A：运行与演示说明 19
附录 B：AI 辅助开发与成本控制 20

摘要

本文提出并实现了一个纯视觉驱动的实时魔法对战系统 Spells Battle。系统以普通 RGB 摄像头作为唯一输入，不依赖手柄、深度相机或穿戴设备，通过 MediaPipe Hands 获得手部 21 个关键点，并结合指尖轨迹或亮色标记魔杖轨迹完成法术识别，最终驱动一套包含法力、冷却、弹体克制、角色增益、PVE 对手与渲染反馈的实时对战逻辑。与常见键鼠对战游戏相比，本项目关注的问题不是单一算法精度，而是如何把不稳定、连续、带噪声的视觉输入转化为低延迟、可解释、可调试、可持续迭代的游戏指令。

项目的主要工作可以概括为四点：第一，构建了基于几何特征的启发式手势分类器，将轨迹方向、跨度、直线度、转向次数、闭合度与面积等特征映射到 Incendio、Stupefy、Glacius、Protego、Lumos、Overload 六类法术；第二，实现了亮色魔杖追踪管线，融合 HSV 颜色标定、动态 ROI、候选打分、稀疏 Lucas-Kanade 光流、alpha-beta 滤波、动态跳变门与 LOCKED/FLOW/COASTING/LOST 四状态跟踪机，用于缓解快速挥动、短暂遮挡和强光干扰；第三，引入 CV Training 数据闭环，记录 intended spell、主启发式预测与 shadow/template 预测，通过日志回放和 DTW 模板训练支持后续分类器升级；第四，将原单体脚本重构为 cv、game、renderer、models 等低耦合模块，并以 35 个测试文件、374 个自动化用例覆盖识别、对战、训练、渲染、设置、PVE 和工具链。

关键词：手势识别；MediaPipe Hands；颜色目标跟踪；光流；alpha-beta 滤波；动态时间规整；实时交互系统；体感游戏

1 相关背景与技术选择

本项目在成熟视觉方法上进行游戏场景适配。MediaPipe Hands 提供单帧手部关键点检测，是食指输入与手掌姿态识别的基础；OpenCV 提供颜色空间转换、形态学处理、轮廓分析与稀疏光流；DTW 可用于比较不同长度的轨迹序列；alpha-beta 滤波则以较低计算成本提供位置与速度的平滑估计。项目在这些方法基础上进行游戏场景适配，并围绕实时性、可维护性和可验证性组织系统实现。

技术	在本项目中的用途	选择原因
MediaPipe Hands	输出 21 个手部关键点，定位食指尖、手掌中心、开掌姿态	本地推理，低成本，适合普通摄像头

HSV 颜色检测	检测亮色魔杖标记，作为 wand 模式的主要观测来源	相比 RGB 更容易处理亮度变化，阈值可解释
Lucas-Kanade 光流	颜色检测短暂失败时进行兜底跟踪	适合短时局部运动估计，计算开销小
alpha-beta 滤波	平滑魔杖点并估计速度，用于预测和跳变门	比完整 Kalman 更轻量，参数更直观
DTW 模板匹配	作为 shadow/template 分类器和后续数据驱动升级路径	可比较不同速度、不同长度的轨迹
启发式几何分类	当前主分类器，基于方向、跨度、转向、闭合度等特征判定法术	零训练成本，失败原因可诊断

2 系统总体架构

系统采用分层模块架构，使视觉输入、游戏规则和渲染表现分别拥有清晰边界：cv/ 负责从摄像头帧产生可解释的轨迹与法术预测；game/ 负责施法、资源、状态、PVE 与模式切换；renderer/ 负责玩家、弹体、HUD、菜单、粒子和后处理效果；models/ 保存玩家、弹体、粒子和轨迹等数据结构。主循环 main.py 将这些模块连接起来：取帧、更新 CV、更新游戏状态、渲染画面、处理输入。

整体数据流可概括为：

> 摄像头帧 -> CV 输入层(指尖/魔杖坐标) -> 轨迹特征提取 -> 手势分类 -> 对战逻辑(施法/克制/伤害) -> 渲染与音频反馈

层次	主要模块	职责
cv/ 视觉输入层	hand_tracker, wand_detector, wand_tracking, gesture_classifier, template_classifier, hybrid_classifier, controller	追踪手部与魔杖，维护 CV 状态机，输出法术预测和调试信息
game/ 游戏逻辑	battle, characters, pve_ai, cv_training,	管理法力、冷却、弹体、克制、角

层	mode, audio, settings	色增益、PVE 对手、训练模式和设置持久化
renderer/ 表现层	player, projectile, effects, postfx, hud, menu, character_assets, text	绘制角色立绘、弹体、HUD、菜单、文字、粒子、震动和后处理
tools/ 离线工具	replay_cv_logs, evaluate_gesture_templates, train_gesture_templates, review_gesture_samples	用于日志回放、分类器评估、模板训练和样本复核
tests/ 自动化验证	35 个 test_*.py 文件	覆盖核心模块、边界条件、兼容路径与工具链

3 视觉输入路径

3.1 Finger 模式：以食指尖作为最小可行输入

finger 模式直接使用 MediaPipe Hands 的 8 号关键点，即食指指尖，作为轨迹采样点。该路径的优势是稳定、直观、实现成本低，适合调试手势分类规则。为了避免偶发丢帧造成轨迹断裂，系统会在短时间内根据上一帧速度进行线性外推，保持不超过 0.14 s 的预测点；同时利用 0/5/9/13/17 五个手掌根部关键点估计手掌中心，并以 p9 与 p0 的距离估算 palm_scale，用于归一化动作尺度和辅助 wand 模式的 ROI 选择。

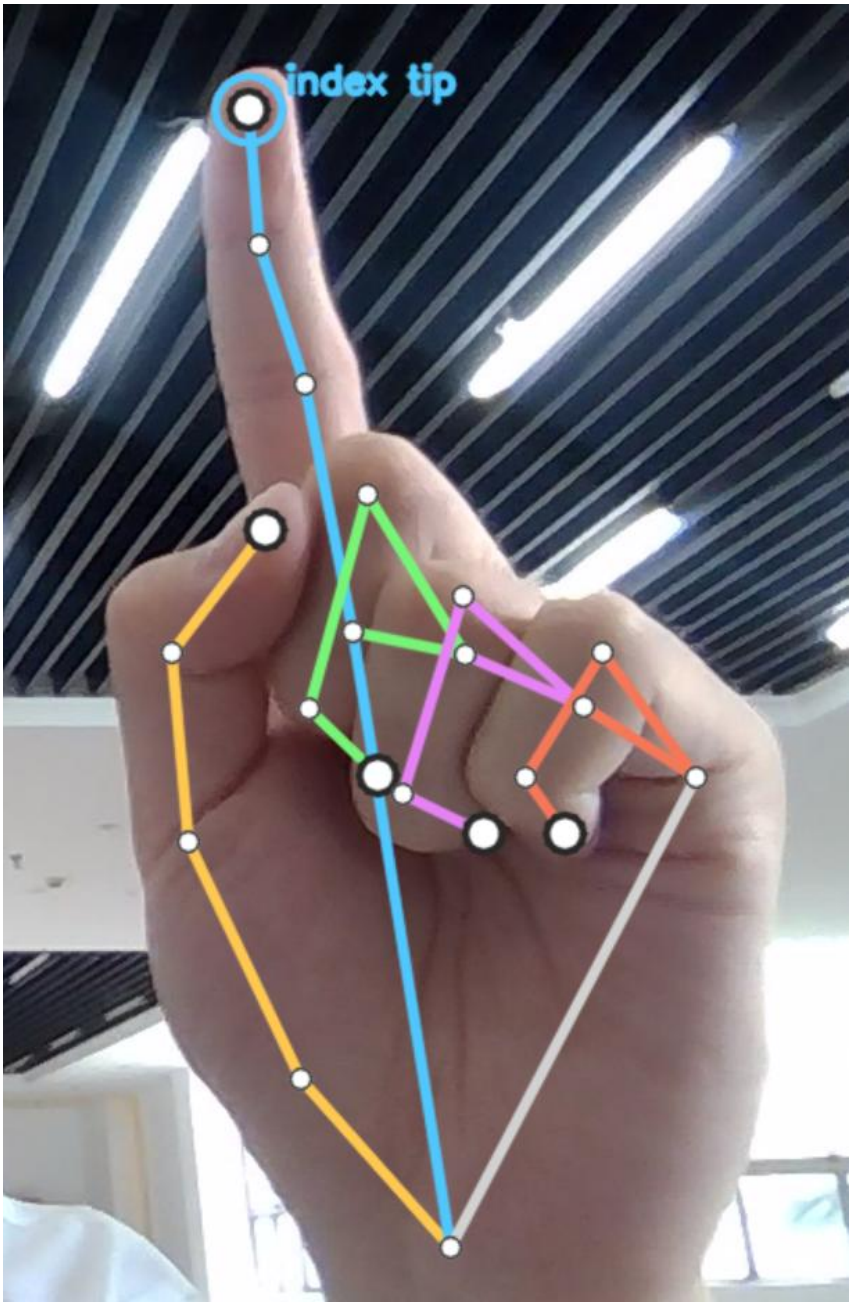


图1 食指尖输入示例：MediaPipe Hands 输出关键点并标记index tip

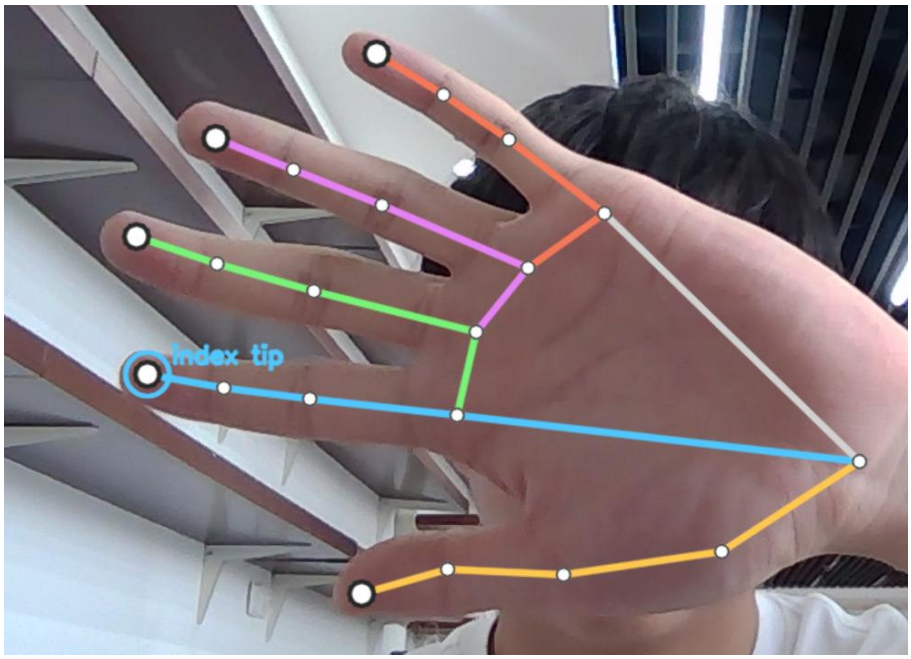


图2 开掌姿态示例：Protego 使用静态手掌姿态判定，区别于轨迹类法术

3.2 Wand 模式：亮色标记魔杖输入

wand 模式在杆尖贴一枚高饱和色块，并通过颜色标定锁定目标色相。相比无标记笔尖，亮色标记更容易在普通摄像头下稳定检出，也更符合项目“低成本但可用”的目标。

项目曾分析普通无标记魔杖路线，但该路线容易受背景边缘、手部轮廓和光照变化影响，且调试成本较高，最终被从 active project 中移除。当前系统只保留 finger 和 bright-marker wand 两条公开输入路径。

3.3 Duel 模式下的多玩家输入归属

双人 Duel 模式需要在同一摄像头画面中同时维护 P1 与 P2 的输入轨迹。MediaPipe Hands 只返回检测到的手部关键点，不直接提供“这只手属于哪位玩家”的语义，因此系统在 cv/controller.py 中先根据手腕横向位置解析玩家索引：当检测到两只手且横向距离超过画面宽度的 12% 时，强制将最左侧手分配给 P1、最右侧手分配给 P2；若距离不足，则回退到半屏规则，并保留更深入该侧的手。该逻辑由 DUEL_FORCE_SPLIT 与 DUEL_SPLIT_MIN_SEPARATION_FRAC 控制，防止两只手落在同一侧时共用同一个 track，导致另一玩家无输入。

wand 模式下还需要处理亮色标记归属。两个玩家使用相近颜色或标记靠近时，单独按 ROI 取最高分候选可能让两个 track 抢到同一候选点。系统通过 _assign_wand_preferred_points 对“手-候选点”

组合做全局分配，评分同时考虑候选置信度、到本方手掌中心的 ownership margin、到上一帧预测点的距离以及 P1/P2 半屏偏好；WAND_ASSIGNMENT_DUPLICATE_RADIUS=30 用于排除重复占用。这样可以将多人输入消歧从局部贪心方式改为一轮小规模全局分配，提升 Duel 模式下的稳定性。

4 亮色魔杖追踪管线

亮色魔杖是本项目最主要的工程难点。它既要比手指输入更有沉浸感，又会引入颜色标定、运动模糊、反光吸附、手部遮挡和快速折返等问题。为了避免简单颜色检测在真实场景中频繁丢失，系统将追踪拆成“颜色观测、候选评分、短时光流、滤波预测、状态机门控”五类职责，并在 debug 日志中记录每次接受或拒绝观测的原因。

4.1 检测与标定

用户进入 wand 模式后，将亮色标记放置在画面中心标定区域并按 C 键。系统在中心小块区域读取 HSV 像素，忽略低饱和度样本，以圆周均值估计主色相，并按分位数自适应设置饱和度与亮度阈值。双人模式下，P1 与 P2 可以分别锁定颜色。

4.2 ROI、候选与评分

每帧检测时，系统首先以手掌中心和上一帧跟踪状态决定搜索 ROI，避免在整幅图像上盲扫颜色。随后对 ROI 内的 HSV 掩码做形态学开闭运算，提取候选 blob。候选点并非只取轮廓中心：对于细长形状，系统还会结合 PCA 端点策略估计可能的杆尖位置。每个候选会根据色相距离、饱和度、明度、面积、紧致度以及到预测位置的距离综合打分，形成可解释的 confidence。

4.3 光流、滤波与状态机

当颜色检测短时失败时，系统启用稀疏 Lucas-Kanade 光流作为兜底，但不会无条件相信光流结果。光流点需要通过颜色一致性二次校验和前后向误差过滤，避免漂移到背景纹理。通过检测或光流得到的观测点会进入 alpha-beta 滤波器，滤波器同时估计位置和速度，使系统能够在短暂遮挡时进行有限预测。最终，追踪状态在 LOCKED、FLOW、COASTING、LOST 之间转移：LOCKED 表示颜色检测可靠，FLOW 表示光流补偿，COASTING 表示短时预测，LOST 表示追踪中断。

状态	来源	作用	风险控制

LOCKED	颜色检测候选	正常输出杆尖点	需要满足检测分数与跳变门
FLOW	LK 光流	颜色短暂失败时兜底	颜色一致性与前后向误差过滤
COASTING	滤波预测	遮挡或丢点时保持极短连续性	时间上限约 0.16 s，避免陈旧预测污染手势
LOST	无可靠观测	终止不可信输入	等待稳定候选或强置信候选重新锁定

4.4 v2 诊断与改进过程

项目的魔杖追踪不是一次写完即稳定，而是经历了基于录屏与日志的诊断。早期问题包括：蓝色贴片肉眼可见但系统长时间显示 NO WAND；快速折返时黄色追踪点滞后在旧位置；起手动作被漏掉，导致轨迹过短或分类依据错误。日志显示，许多失败并非系统完全看不见候选，而是候选分数略低于 relock 阈值，或真实快速运动被固定 jump gate 当作异常跳点拒绝。

因此 v2 版本引入了候选历史、稳定低分重锁、动态跳变门和更具体的 debug 文案。候选历史记录最近数帧的候选点，若低分候选在短时间内稳定出现在相近位置，则允许重新锁定；动态 jump limit 根据速度、丢失时长和候选分数放宽快速运动下的位移限制；当候选被拒绝时，debug 不再只显示 no bright tip，而会给出 low_score、jump、limit、relock stable 等更具体的原因。这些设计的目标是：在真实可见标记出现时尽快恢复，同时避免旧预测点长期误导手势分类。

5 手势分类算法

5.1 几何特征启发式分类

当前主分类器使用几何启发式。系统从轨迹点列中提取首尾位移 dx/dy、包围盒 x_span/y_span、轨迹总长 path_len、直线度 straightness、转向次数 turn_count、面积 area、圆度 circularity 等特征，再将其与各法术的动作语义匹配。与黑箱模型相比，这种方法的优点是阈值可调、速度快、失败原因可诊断；缺点是对用户动作差异较敏感，需要通过日志和训练数据逐步校准。

法术	动作语义	主要判定特征
Incendio	朝对手方向水平直推	opponent_dx 大，straightness 高，横向位移明显
Stupefy	闭合圆圈	circularity 高，闭合距离较小，面积足够
Glacius	向下劈砍	dy 大，横向跨度较窄，路径较直
Protego	张开手掌保持	基于手掌姿态和静止保持，不依赖轨迹 DTW
Lumos	向上挑	dy 为负，纵向跨度大，路径长度足够
Overload	多段锯齿闪电	turn_count 较高，方向反转明显

为了处理规则冲突，系统收集所有候选匹配并通过 score_gesture_match 排序，select_best_match 选择得分最高者，并对锯齿与直线等易冲突场景做额外加权。该策略保留了启发式规则的可解释性，也减少了某些动作被错误优先级抢占的问题。

Protego 的开掌检测还加入了速度门控。should_consume_open_palm 在开掌置信度满足要求后，会继续计算手部当前位置相对上一帧的移动速度；当速度超过 CV_PALM_MAX_CAST_SPEED=360.0 px/s 时，本次开掌不会被判定为护盾施法，而会被视为移动过程中的姿态瞬间。该门控使 Protego 更接近“保持开掌”的交互语义，减少轨迹类法术挥动过程中误触发护盾。

5.2 DTW 模板与 shadow 评估

DTW 用于比较不同长度、不同速度的轨迹序列。项目中 DTW 一开始并未直接参与分类，而是作为 template/shadow 管线记录旁路预测。这样做的原因是：真实用户数据不足时，模板分类器很可能被少量样本或合成轨迹误导；如果让它直接施法，会影响我们测试时期的游戏稳定性，进而影响我们对启发式分类方法的效果评估。因此系统先记录 intended spell、主启发式预测和 shadow/template 预测，再通过 replay_cv_logs 和 evaluate_gesture_templates 离线比较。

5.3 FusedGestureClassifier 融合策略

在已有模板模型时，系统采用决策树式融合。早期加权思路中，若 heuristic 权重为 1.0、DTW 权重为 0.2，DTW 即便给出高分也很难纠正启发式错误，实际校正作用较弱。FusedGestureClassifier 的融合策略基于语义分支决策：二者给出同一法术时，将 confidence 提升 max(heuristic.confidence, template.confidence) + 0.08；二者冲突时，若 template.confidence >= 0.70 且不低于 heuristic.confidence - 0.05，则允许模板结果覆盖启发式结果；启发式没有结果时，若 template.confidence >= 0.75，则使用模板进行 DTW 救场；其余情况下保留启发式结果。该策略通过 TEMPLATE_OVERRIDE_MIN、CONFLICT_MARGIN、TEMPLATE_RESCUE_MIN 和 AGREE_BOOST 明确控制模板参与主决策的边界。

```
if heuristic.spell and template.spell and heuristic.spell == template.spell:
    confidence = max(heuristic.confidence, template.confidence) + 0.08
    return heuristic.spell with boosted confidence

if heuristic.spell and template.spell and template.spell != heuristic.spell \
   and template.confidence >= 0.70  # TEMPLATE_OVERRIDE_MIN
   and template.confidence >= heuristic.confidence - 0.05:  # CONFLICT_MARGIN
    return template.spell

if heuristic.spell is None and template.spell \
   and template.confidence >= 0.75:  # TEMPLATE_RESCUE_MIN
    return template.spell

if heuristic.spell:
    return heuristic.spell

return None
```

6 CV Training 数据闭环

CV Training 模式是分类器迭代升级的基础。它的价值在于把真实操作中的识别偏差转化为可回放、可标注、可比较的数据。正常对战中，系统不会打断玩家要求标注；训练模式则专门冻结完成的轨迹，让用户确认、丢弃或重新标注 intended spell。每条训练样本会记录轨迹点列、意图法术、主启发式预测、模板/DTW 预测，以及必要的上下文信息。

数据闭环的流程如下：

采集真实轨迹样本 -> 打标/复核 intended spell -> 记录 heuristic + template 预测 -> 回放日志离线对比 -> 证明更优后升级分类器

机制	具体实现	意义
独立采集模式	模式 4 CV Training 冻结完整轨迹供复核	不干扰正常对战体验
三路记录	intended_spell、primary prediction、shadow/template prediction	支持客观比较而非凭感觉调参
断点续采	progress.json 保存已接受样本数	返回菜单后可继续采集
离线回放	tools/replay_cv_logs.py 汇总预测一致性	定位误判集中在哪些法术对
模板训练	tools/train_gesture_templates.py 从复核样本训练 gesture_templates.json	让 DTW 从真实数据学习

这种设计使分类器升级具有可验证路径。只有当真实训练日志表明某个法术的 template/shadow 预测持续优于启发式，才考虑让该法术局部交给模板分类器。这比一次性替换主分类器更稳健，也符合实时交互系统对误触风险的要求。

7 对战逻辑与玩法系统

游戏逻辑集中在 game/battle.py。系统维护法力、冷却、施法状态、弹体碰撞、伤害、护盾、沉默、冰冻、治疗和胜负状态。玩家法力上限为 100，基础回复为 12/s；系统设有 1.0 s 全局冷却和每个法术的独立冷却，避免连续误触造成大量法术堆叠。施法失败时会返回明确原因，例如 NO MANA、CD、GCD、SILENCE、DEAD 或 ROUND OVER，这些原因既用于 HUD 提示，也用于测试断言。

法术	资源/冷却	效果

Incendio	25 mana，3.0 s	火系弹体，基础攻击法术
Stupefy	35 mana，5.0 s	昏迷弹体，命中后造成沉默
Glacius	30 mana，4.6 s	冰系弹体，造成减速/冻结效果
Protego	30 mana，6.0 s	限时护盾，吸收伤害
Lumos	40 mana，8.0 s	吟唱治疗，并提升短时法力回复倍率
Overload	至少 45 mana，消耗全部法力	高风险高收益弹体，命中后禁用目标增益

弹体克制采用简化环：Incendio 克制 Stupefy，Stupefy 克制 Glacius，Glacius 克制 Incendio；未列出的组合通常双方湮灭并触发音效。这样的规则让手势识别不仅是输入技术，也能进入玩法策略。角色系统提供 Harry、Hermione、Ron、Snape 四个角色：Harry 提升弹体伤害，Hermione 缩短冷却，Ron 提升最大生命值，Snape 的弹体在普通反制中更不容易被清除。PVE 模式使用规则型对手，可攻击、护盾和治疗，并包含可见蓄力，使玩家能够读招应对。

8 渲染表现与交互反馈

渲染层不仅承担画面输出，也承担实时交互中的“解释反馈”。系统绘制玩家、弹体、粒子特效、屏幕震动、命中 flash、HUD、菜单和角色选择界面；当 CV debug 打开时，还会显示当前输入模式、追踪状态、轨迹点数、分类置信度和拒绝原因。为了降低资源依赖，缺失立绘时系统回退到代码自绘玩家，音频不可用时保留合成兜底，中文路径下图片读取则使用 np.fromfile + cv2.imdecode 规避 cv2.imread 对中文路径的不稳定支持。



图3 多角色多状态立绘资源：normal / casting / hit / win / lose

多状态立绘、粒子、后处理、屏幕震动和音效同时也是对战反馈的一部分：施法动作、弹体碰撞、护盾格挡、角色受击、胜负状态都需要及时反馈，否则玩家难以判断自己的手势是否被识别、法术是否被冷却阻挡、弹体是否被克制。

渲染层的后处理由 renderer/postfx.py 中的 PostFX 统一完成。系统在法术层绘制前保存快照。绘制后通过 cv2.subtract 提取新增亮区，并将其注入带指数衰减的 trail 缓冲（trail_decay=0.82）。随后系统将 trail 缓冲叠加回当前帧，形成法术拖尾。辉光部分使用灰度阈值 bloom_threshold=170 提取亮区。该亮区先经过掩码和 8 倍下采样，再进行高斯模糊和上采样。最后以 bloom_strength=0.9 加法叠加到画面。该“快照-差值-衰减缓冲”的方式无需为每个弹体单独维护拖尾历史，适合实时对战中大量短生命周期特效。

音频反馈采用外部资源优先、程序化合成兜底的策略。game/audio.py 会优先加载 assets/audio/ 中的 wav/ogg；资源缺失时，使用 44100 Hz 双声道 NumPy 波形生成施法、命中、护盾、失败、菜单、回合结束和环境循环等音效。不同法术对应不同合成特征，例如 Incendio 使用频率扫描与锯齿波，Protego 叠加 280/560/840 Hz 三层正弦与 shimmer。这样在缺少素材或部署环境变化时，游戏仍能保留关键听觉反馈。

9 工程实现与可维护性

9.1 从单体脚本到模块化结构

项目初期以单体脚本为主，进行快速验证玩法和模型迭代，但随着 CV 状态机、PVE、训练模式、角色系统和渲染资源增加，继续堆叠在单文件中会使调参和回归测试困难。因此项目将主逻辑拆分为职责单一的模块，并保留 legacy/ 目录作为历史参考。该重构让不同工作可以独立推进：调魔杖追踪不需要改 battle，调 PVE 不需要碰 renderer，改 HUD 不影响 template classifier。

9.2 参数集中与兼容回退

系统的核心阈值集中在 config.py，例如 WIDTH=1280、HEIGHT=720、FPS fallback=60、CV_TRACK_TIMEOUT=0.65、CV_PREDICT_HOLD_TIME=0.14、CV_MAX_MISSED_FRAMES=3、WAND_MIN_SATURATION=70、WAND_MIN_VALUE=70、WAND_HUE_TOLERANCE=12 等。集中配置使实验可以复现，也方便报告中明确说明当前设置。回退机制包括无摄像头时生成程序化背景、缺失立绘时绘制几何玩家、旧设置 plain_wand 回退到 finger、音频文件缺失时使用合成音效。

文字渲染也采用双路径回退：renderer/text.py 优先使用 PIL 字体路径，缓存文字位图并支持中文字体回退；PIL 不可用时转为 cv2.putText + FONT_HERSHEY_SIMPLEX，同时保持 baseline 坐标约定一致，使 HUD 与菜单布局不依赖具体文字后端。

9.3 工具链

工具脚本	作用
tools/replay_cv_logs.py	读取 CV session 日志，比较 intended、 primary 和 shadow/template 预测
tools/evaluate_gesture_templates.py	评估启发式、模板和融合分类器准确率，支持对比实验
tools/train_gesture_templates.py	从复核样本训练数据驱动 gesture_templates.json
tools/review_gesture_samples.py	提供样本复核入口，辅助修正训练标签
tools/retrain_gesture_templates.py	串联复核样本与模板重训练流程

10 测试与验证

项目使用 unittest 进行自动化验证。当前仓库包含 35 个 test_*.py 文件，覆盖对战规则、角色增益、CV 状态机、魔杖检测、魔杖追踪、模板分类器、训练模式、PVE AI、渲染文本、HUD、菜单、设置持久化、音频、工具脚本和 smoke tests。自动化测试保证每次修改后基础行为不回归。手动演示仍用于验证交互体验。

```
python -m compileall -q .
python -m unittest discover -s tests -v
```

在 conda Python + OpenCV 项目环境中执行全量测试，结果为：Ran 374 tests in 8.441s OK。该结果覆盖当前 35 个 test_*.py 文件对应的自动化用例，作为本报告的测试基线。

测试类别	覆盖重点
CV 状态机	IDLE/RECORDING/CLASSIFY/BLOCKED 等状态转移，冷却/沉默阻挡后的轨迹清理

魔杖检测	颜色标定、低饱和拒绝、候选历史、稳定重锁、动态跳变门、LK 兜底
分类器	几何分类、DTW 距离、模板加载、融合 rescue、一致性奖励
游戏规则	法力、冷却、弹体克制、护盾、治疗、Overload 增益禁用
训练工具	样本保存、断点续采、模板训练、离线评估、复核流程
渲染与设置	角色立绘回退、文字绘制、HUD、菜单布局、设置兼容

除自动化测试外，项目还需要手动 CV 评估。建议按 finger 和 wand 分开测试，每种法术各执行 10 次，记录 Correct、WrongSpell、NoTrigger、Blocked 和 CommonReason。对于 wand 模式，还应记录标记颜色、光照条件、相机距离、是否校准、是否靠近掌心或画面边缘。只有在真实操作记录中达到目标正确率后，才适合声称某条输入路径 demo-ready。

11 实验结果与分析

11.1 实时性

系统目标运行在 1280x720 分辨率和 60 FPS 反馈节奏下。为了控制开销，颜色检测主要限制在手掌邻域或预测区域内，而不是全图扫描；MediaPipe Hands 负责稳定输出手部关键点；魔杖追踪在颜色检测失败时只做短时光流和短时预测。这种设计把计算资源集中在最可能出现输入的位置，符合实时游戏的需求。

11.2 魔杖鲁棒性

从诊断材料看，魔杖问题主要集中在重锁慢、固定 jump gate 过严和 coasting 输出旧预测三个方面。v2 改进后的测试覆盖了低分候选稳定重锁、强置信远距离重锁、快速连续运动接受、白光干扰拒绝、短时 coasting 终止等场景。自动化测试能够证明这些边界行为符合预期，但最终鲁棒性仍需要结合录屏和手动评估确认。

11.3 分类融合策略

启发式分类器的优势是可解释和稳定，DTW/template 的优势是有潜力从真实轨迹中学习。早期加权求和方案会让 DTW 因权重过低而难以发挥校正作用；当前决策树融合为 DTW 赋予“启发式放弃时救场”和“一致时增加置信”的明确语义。这种策略更适合项目当前阶段：保持现场演示稳定，同时给未来数据驱动升级留下入口。

维度	加权求和方案	决策树融合方案
DTW 角色	权重低，难以纠错	启发式无结果时救场，高一致性时加信心
决策语义	分数叠加，不易解释	每个分支含义明确
风险控制	可能受权重设置影响	冲突时保留启发式，较保守
升级路径	难判断 DTW 何时真正有效	可结合日志逐步开放某些法术

12 已知不足与未来工作

当前系统已经形成完整可玩的原型，但仍存在若干不足。第一，亮色魔杖在强光、低饱和标记或快速运动下仍可能短时丢失，后续需要继续调优饱和度门限、候选评分和重锁策略。第二，分类器仍以启发式为主，DTW/template 只有在部分条件下参与融合，真正让数据驱动分类器接管还需要更多真实标注样本和回放评估。第三，HUD 信息密度仍偏高，演示时有助于调试，但正式游玩时可能干扰玩家注意力。第四，角色立绘和部分视觉资源仍带占位性质，需要后续统一美术风格。

后续工作可按以下优先级推进：

- 前端表现整理：将教学说明移到菜单和训练界面，降低对战 HUD 的实时信息密度。
- 继续稳定 bright-marker wand：重点关注强光误吸附、快速折返和起手帧漏检。
- 扩大 CV Training 样本：分别采集 finger 和 wand 轨迹，形成可回放数据集。
- 基于日志评估模板分类器：按法术统计启发式、模板和融合策略的准确率和混淆矩阵。
- 在证据充分时进行局部 handoff：只让 shadow 明显优于 heuristic 的法术逐步进入主分类器。

13 结论

本文实现了一个仅依赖普通摄像头的实时魔法对战系统。项目将连续、噪声较大的视觉输入接入到一个可玩的实时游戏系统中，并围绕稳定性、可解释性和可持续迭代建立了完整工程链路。MediaPipe Hands 提供了可靠的手部关键点基础，亮色魔杖追踪管线解决了沉浸输入中的关键稳定性问题，几何启发式分类器保证了低成本和可诊断性，DTW/template 与 CV Training 则为未来数据驱动升级提供路径。通过模块化重构、自动化测试和离线工具链，系统从单体 MVP 演进为一个更可维护的课程项目原型。

参考文献

1. F. Zhang, V. Bazarevsky, et al. MediaPipe Hands: On-device Real-time Hand Tracking. arXiv:2006.10214, 2020.
2. B. D. Lucas, T. Kanade. An Iterative Image Registration Technique with an Application to Stereo Vision. Proc. IJCAI, 1981.
3. D. S. Bolme, J. R. Beveridge, et al. Visual Object Tracking using Adaptive Correlation Filters. Proc. CVPR, 2010.
4. J. O. Wobbrock, A. D. Wilson, Y. Li. Gestures without Libraries, Toolkits or Training: A $1 Recognizer for User Interface Prototypes. Proc. UIST, 2007.
5. H. Sakoe, S. Chiba. Dynamic Programming Algorithm Optimization for Spoken Word Recognition. IEEE Trans. ASSP, 1978.
6. G. Bradski. The OpenCV Library. Dr. Dobb's Journal of Software Tools, 2000.
7. R. E. Kalman. A New Approach to Linear Filtering and Prediction Problems. ASME Journal of Basic Engineering, 1960.

附录 A：运行与演示说明

```
python main.py
```

- 主菜单 1/2/3/4 分别进入 Solo Spell Showcase、 Duel Battle、 PVE Battle、 CV Training。
- V 在 finger 与 wand 输入模式之间切换；wand 模式下把亮色标记放到中心校准区域并按 C。
- D 切换 compact / verbose CV debug overlay；SPACE 重置当前回合；M 返回菜单。
- CV Training 中 Enter 保存冻结样本，Backspace 或 Space 丢弃样本，数字键可重新标注意图法术。
- B 开关 BGM

附录 B：AI 辅助开发与成本控制

项目开发过程中，AI 辅助主要用于代码审阅、重构建议、测试补充、报告结构整理和故障诊断，而不是直接替代手动验收。为了控制成本，交互时尽量提供最小必要上下文，例如只给相关模块、错误日志和期望行为；长文档先整理结构提纲，再逐章扩写；代码修改后用自动化测试和手动演示验证，减少让模型在缺乏实际验证的情况下给出猜测性答复。这样既能提升迭代效率，也能避免在长对话中反复输入无关内容。